

Artificial Intelligence for Video Games

Chasing Flock

Zong Yuan Alessandro Gong

19167A

Project description

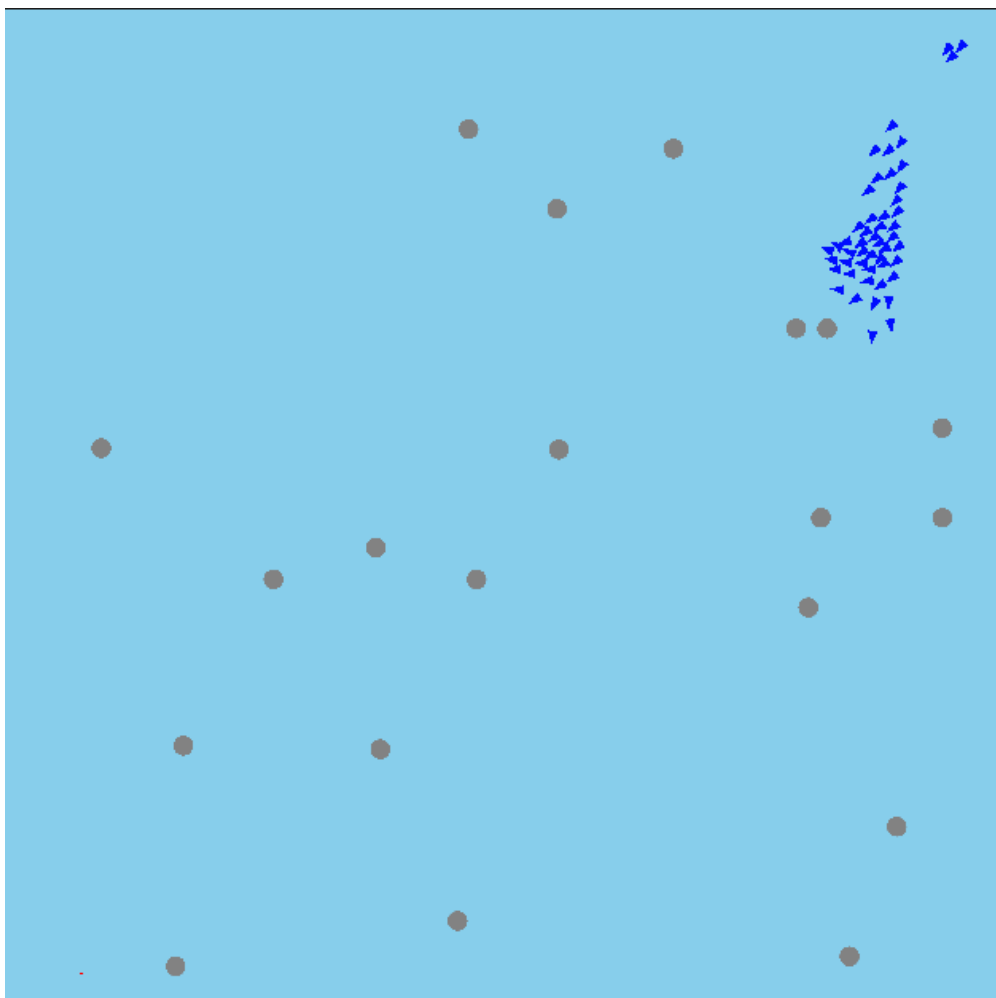
The project simulates a flock of agents moving inside a scene characterized by moving obstacles. The flock must move toward a target randomly placed in the scene while avoiding the obstacles.

The scene is a 100x100 square, acting as boundary for the flock.

The flock is composed of 50 agents moving at 10 m/s, following the Craig Reynolds approach.

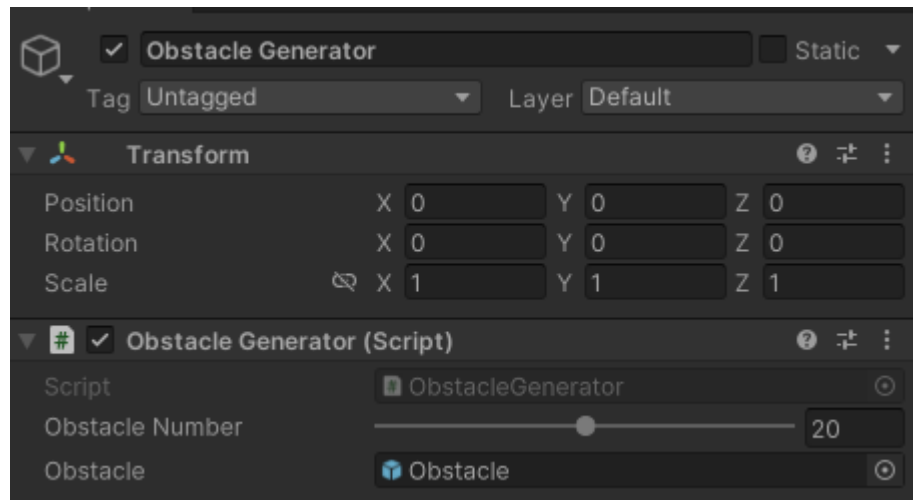
In the scene, there are from 10 to 30 obstacles and each one of them move back and forth horizontally at speeds between 5 and 20 m/s.

The target is randomly placed nearby one of the corners of the scene and whenever one of the agents of the flock reaches it, its position is randomized again.



Obstacles

Obstacles are generated at the start of the scene by the Obstacle Generator game object. From the GUI it's possible to select the number of obstacles generated in the scene.



When an obstacle is instantiated into the scene, a random speed between 5 and 20 m/s is assigned to it. When colliding with a wall, their direction flips to remain inside the scene.

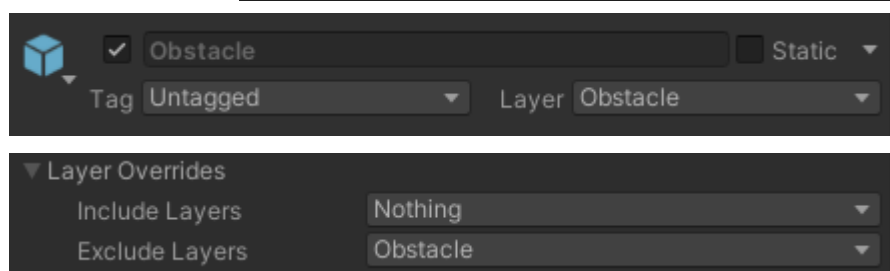
Obstacle do not collide with each other. To achieve such thing, a layer is assigned to them. Such layer is then ignored when checking for collisions.

```
public class ObstacleController : MonoBehaviour
{
    2 references
    private int speed = 0;
    4 references
    private Vector3 dir;

    0 references
    void Start() {
        speed = Random.Range(5, 21);
        dir = new Vector3(Random.Range(0,2) * 2 - 1, 0, 0);
    }

    0 references
    void FixedUpdate() {
        transform.position += dir * speed * Time.deltaTime;
    }

    0 references
    void OnCollisionEnter2D(Collision2D collisionInfo) {
        if (collisionInfo.gameObject.layer == 6) {
            dir = Vector3.Reflect(dir, Vector3.right);
        }
    }
}
```



Target

The target position is randomized at the start of the scene with the function `ResetPosition()` which randomly selects one of the corners of the scene (excluding the current corner) and places the target at a maximum distance of 10 meters from it.

```
void ResetPosition() {
    int newCorner = Random.Range(1, 5);
    while (newCorner == corner) {
        newCorner = Random.Range(1, 5);
    }

    corner = newCorner;
    Vector3 cornerPos = Vector3.zero;
    Vector2 tmpPos = Random.insideUnitCircle;
    Vector3 targetPos = Vector3.zero;

    switch (corner) {
        case 1:
            cornerPos += new Vector3(-50, 50);
            targetPos += new Vector3(Mathf.Abs(tmpPos.x), -Mathf.Abs(tmpPos.y));
            break;
        case 2:
            cornerPos += new Vector3(50, 50);
            targetPos += new Vector3(-Mathf.Abs(tmpPos.x), -Mathf.Abs(tmpPos.y));
            break;
        case 3:
            cornerPos += new Vector3(50, -50);
            targetPos += new Vector3(-Mathf.Abs(tmpPos.x), Mathf.Abs(tmpPos.y));
            break;
        default:
            cornerPos += new Vector3(-50, -50);
            targetPos += new Vector3(Mathf.Abs(tmpPos.x), Mathf.Abs(tmpPos.y));
            break;
    }

    transform.position = cornerPos + targetPos * radius;
}
```

The target has a Trigger collider which gets activated when one of the flock agents touches it, causing the position to be reset.

```
void OnTriggerEnter2D(Collider2D other) {
    if (other.gameObject.layer == 8) {
        ResetPosition();
    }
}
```

Flock

The flock is generated by a Spawner game object which holds the parameters of the flock. The Spawner generates the agents in a circular area and makes sure to select a position not overlapped with other elements in the scene.

```
void Start () {  
    if (boid != null) {  
        for (int i = 0; i < count; i++) {  
            bool creating = true;  
            Vector3 boidPos = Vector3.zero;  
            while (creating) {  
                Vector3 tmpPos = Random.insideUnitCircle;  
                Vector3 testPos = transform.position + tmpPos * radius;  
                Collider2D collision = Physics2D.OverlapCircle(testPos, 1.0f);  
                if (!collision) {  
                    creating = false;  
                    boidPos += testPos;  
                }  
            };  
  
            GameObject go = Instantiate(boid, boidPos, transform.rotation);  
            go.transform.Rotate(0.0f, 0.0f, Random.Range(0.0f, 359.9f));  
            go.name = boid.name + " " + i;  
        }  
    }  
}
```

The flock agents follow the original approach of Craid Reynolds, that combines the three behaviours of Alignment, Cohesion and Separation to handle the flock movement. The direction of the movement follows the up vector of the agents.

Alignment

```
public class BoidAlign : BoidComponent {  
    0 references  
    override public Vector3 GetDirection(Collider2D[] neighbors, int size) {  
        Vector3 alignment = Vector3.zero;  
        for (int i = 0; i < size; i++) {  
            if (neighbors[i].gameObject.layer == gameObject.layer) {  
                alignment += neighbors[i].gameObject.transform.up;  
            }  
        }  
        return alignment.normalized * BoidShared.AlignComponent;  
    }  
}
```

This component makes sure the agent follows the direction of the agents nearby.

Cohesion

```
public class BoidCohesion : BoidComponent {  
    0 references  
    override public Vector3 GetDirection(Collider2D[] neighbors, int size) {  
        Vector3 cohesion = Vector3.zero;  
        float counter = 0f;  
        for (int i = 0; i < size; i++) {  
            if (neighbors[i].gameObject.layer == gameObject.layer) {  
                cohesion += neighbors[i].transform.position;  
                counter += 1f;  
            }  
        }  
        cohesion /= (float) counter;  
        cohesion -= transform.position;  
        return cohesion.normalized * BoidShared.CohesionComponent;  
    }  
}
```

This component makes sure the agent doesn't move away too much from the centre of mass of the agents nearby.

Separation

```
public class BoidSeparation : BoidComponent {  
    0 references  
    override public Vector3 GetDirection(Collider2D[] neighbors, int size) {  
        Vector3 separation = Vector3.zero;  
        Vector3 obstacle = Vector3.zero;  
        Vector3 wall = Vector3.zero;  
        Vector3 tmp;  
        for (int i = 0; i < size; i++) {  
            Vector3 closestPoint = neighbors[i].ClosestPoint(transform.position);  
            tmp = transform.position - closestPoint;  
  
            if (neighbors[i].gameObject.layer == gameObject.layer) {  
                separation += tmp.normalized / (tmp.magnitude + 0.0001f);  
            } else if (neighbors[i].gameObject.layer == 7) {  
                obstacle += tmp.normalized / (tmp.magnitude + 0.0001f);  
            } else if (neighbors[i].gameObject.layer == 6) {  
                wall += tmp.normalized / (tmp.magnitude + 0.0001f);  
            }  
        }  
        return separation.normalized * BoidShared.SeparationComponent +  
            obstacle.normalized * BoidShared.ObstacleComponent +  
            wall.normalized * BoidShared.WallComponent;  
    }  
}
```

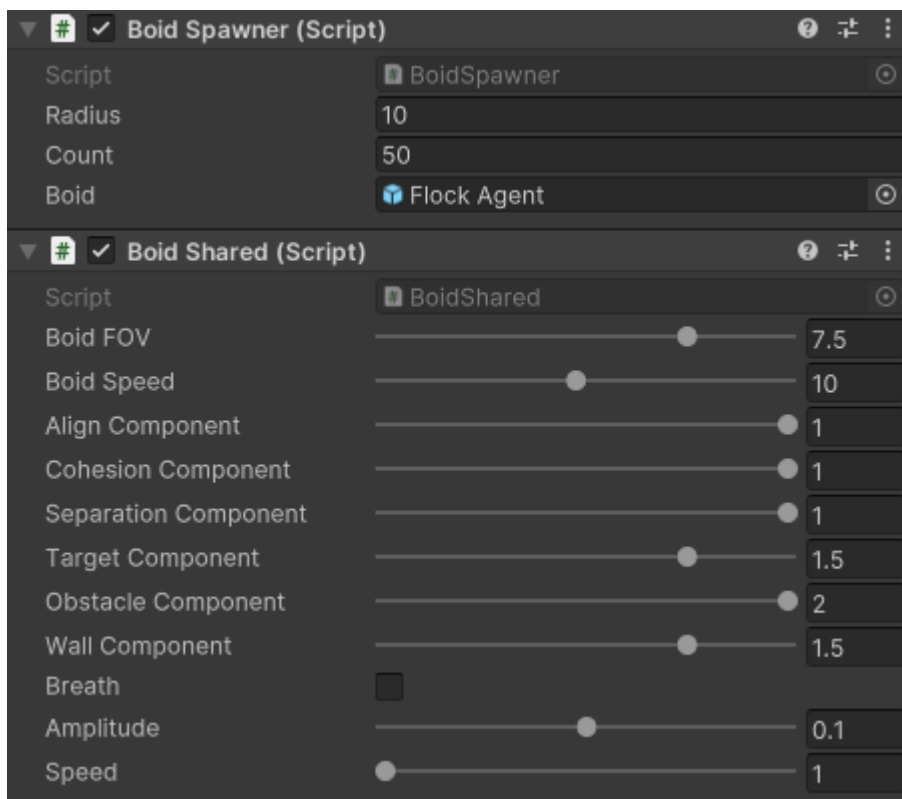
The separation component separates the different game objects that each member of the flock must avoid. Separation is calculated for other members of the flock, the moving obstacles and the walls of the scene. Each element deserves its own weight to reach the best results.

Blending

```
public class BoidBlending : MonoBehaviour {  
    3 references  
    public GameObject target = null;  
  
    2 references  
    private Collider2D[] neighbors = new Collider2D[100];  
  
    0 references  
    void Start() {  
        target = GameObject.Find("Target");  
    }  
  
    0 references  
    void FixedUpdate() {  
        Vector3 globalDirection = Vector3.zero;  
  
        if (target != null) {  
            Vector3 targetDir = target.transform.position - transform.position;  
            globalDirection += targetDir.normalized * BoidShared.TargetComponent;  
        }  
  
        int count = Physics2D.OverlapCircleNonAlloc(transform.position, BoidShared.BoidFOW, neighbors);  
  
        foreach (BoidComponent bc in GetComponents<BoidComponent>()) {  
            globalDirection += bc.GetDirection(neighbors, count);  
        }  
  
        if (globalDirection != Vector3.zero) {  
            transform.rotation = Quaternion.LookRotation(Vector3.forward, (globalDirection.normalized + transform.up) / 2f);  
        }  
  
        transform.position += transform.up * BoidShared.BoidSpeed * Time.deltaTime;  
    }  
}
```

The blending function first calculates the direction to reach the target and then combines it with the directions suggested by the other components to establish the direction the agent should be moving.

Flocking parameters



The flock spawns inside a circle of radius 10 meters. This allows the agents to spawn far enough from each other without being too sparse on the scene.

The FOV has been set to 7.5 meters, as it's necessary for the agents to see incoming obstacles early enough to move away from them. Setting it to a lower value causes the agents to be unable to move away from obstacles moving at faster speeds (since the fastest obstacles move twice as fast as the agents). It also cannot be too high as the flock would be affected too much by obstacles too far away that were not going to hit it.

The alignment, cohesion and separation component must have all high value to avoid being overpowered by the other 3 components added.

The obstacle component is set at 2 since it represents the most critical object that the agents must be aware of. A value too low will cause the agents to often ignore the incoming fast obstacle and collide with it.

Target component has a value of 1.5 because of the high obstacle component. A value too low would make the agent being stuck at dodging the obstacles without moving closer to the target. This becomes extremely noticeable when

the target is very close to the walls with close obstacles. In this situation, the flock would be stuck moving in circles in the nearby area instead of approaching the target.

Wall component has been added at 1.5 to make sure the flock direction doesn't get overpowered by the obstacle and target component, causing it to hug the wall instead of avoiding it. This value cannot be too much lower than the target component and cannot be higher than it to avoid having it stop the flock from reaching targets close to the walls.

Problems and possible improvements

Because of how high the last 3 components are, it's likely that the members of the flock end up overlapping with each other in the attempt to avoid incoming obstacles. A possible solution to this could be to use different weights depending on how close and how fast the detected obstacles are. This way you could prioritize avoiding fast approaching obstacles and moving away from each other when slower ones are close.

A problem that might appear is the breaking of the flock. Because of large groups of obstacles that spawned close to each other, members of the flock could be inclined to move away from the flock to avoid them and being separated. This is partially mitigated by the presence of a target all members of the flock move towards to, but it can be problematic without one. Possible solution could be to increase the FOV of the agents and increase the separation, so that agents stay farther away from each other, giving them plenty of space to avoid obstacles and rejoin the flock.